Università degli Studi di Padova

Dipartimento di Matematica

Laurea Triennale in Informatica

Progetto di Basi di Dati

Basi di Dati per la Gestione di una Rete Forroviaria

Riccardo Ceccato, Stefano Maso

.

Indice

1 Abstract **2**

2 Analisi dei Requisiti **2**

3 Progettazione Concettuale **3**

4 Progettazione Logica **5**

4.1 Eliminazioni delle generalizzazioni 5

4.2 Schema relazionale 6

5 Implementazione in PostgreSQL e Definizione delle query **7**

5.1 Definizione delle query 7

5.2 Creazione degli indici 10

6 Applicazione Software **11**

1 Abstract

Questo lavoro sviluppa una base di dati progettata per gestire in modo strutturato e coerente le informazioni relative ai treni, alle stazioni, ai controllori, alle tratte, agli utenti e biglietti comprati da ciascun utente.
La base di dati proposta distingue tra utenti comuni e controllori in grado di verificare e validare i biglietti. Le tratte sono descritte in termini di stazione di partenza e arrivo e le varie fermate che compongono una determinata tratta. Il sistema tiene inoltre traccia della data in cui i treni, regionali o ad alta velocità, ciascuno con attributi specifici, percorrono le tratte.
Questa base di dati è progettata per garantire un'archiviazione efficiente e strutturata delle informazioni relative alle corse dei treni.

2 Analisi dei Requisiti

Questa sezione riassume i requisiti a cui deve sottostare la base di dati.

Stazioni. Ogni stazione è identificata dal proprio nome e contiene le seguenti informazioni:

- **Nome** che distingue in modo univoco ogni stazione.
- **Città** in cui si trova.
- **Regione** in cui si trova.

Treni. Ogni treno è identificato da un codice univoco ed è caratterizzato dai seguenti attributi:

- **Codice univoco** per distinguere ogni treno.
- **Posti business** disponibili di ogni treno.
- **Posti C1** (classe 1) disponibili in ogni treno.
- **Posti C2** (classe 2) disponibili in ogni treno.
- **Modello** del treno.

I treni si dividono in: **regionali** e **alta velocità**.

Tratte. Ciascuna tratta viene identificata per semplicità da un codice univoco e possiede le seguenti informazioni:

- **Codice univoco** per riconoscere ogni tratta.
- **Distanza** in km della tratta.
- **Durata** in minuti della tratta.

Utenti. Ciascun utente viene identificato dal proprio codice fiscale e ha i seguenti attributi:

- **Codice fiscale** per essere identificato.
- **Nome**.
- **Cognome**.
- **Email**.

Un utente può anche essere un **controllore** o un **cliente**.

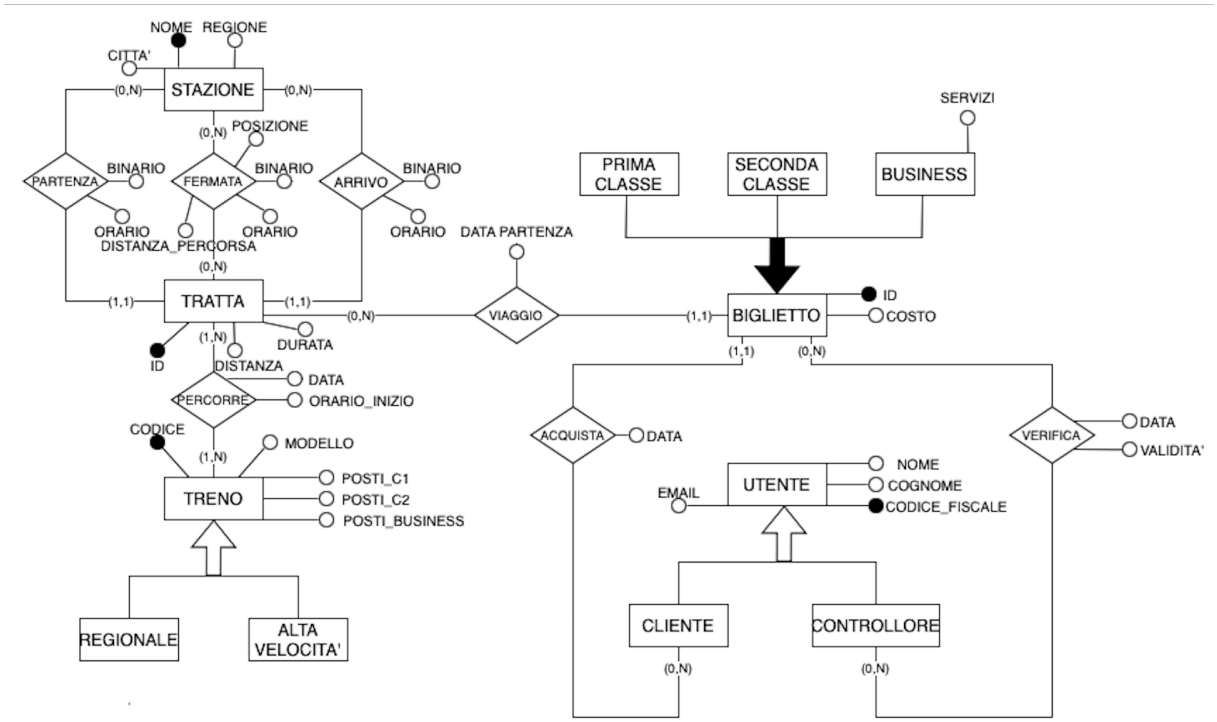
Biglietti. Ogni biglietto ha un codice per garantire l'univocità e possiede gli attributi che seguono:

- **Codice univoco**.
- **Costo** di ciascun biglietto.

I biglietti possono essere di diversi tipi: **business**, **prima classe** e **seconda classe**, solo i biglietti di prima classe hanno un attributo specifico che riguarda la presenza o meno di servizi.

3 Progettazione Concettuale

Il diagramma Entità-Relazione che segue riassume i requisiti descritti nella sezione 2.



Il controllore è un utente speciale, che è in grado di verificare i biglietti, andando a memorizzare il suo stato (valido o non valido) e la data in cui avviene la verifica del biglietto. Ciascun biglietto potrà essere verificato da più controllori.
I treni vengono identificati mediante un codice univoco di otto caratteri. Esistono due tipi di treni, quelli regionali e quelli ad alta velocità; nonostante ciò, non ci sono informazioni specifiche per nessuno dei due tipi. Allo stesso modo, siamo in grado di distinguere diversi tipi di biglietti: business, prima classe e seconda classe; solo i biglietti "business" hanno un attributo specifico che riguarda la presenza o meno di servizi.
L'attributo Distanza_Percorsa della relazione Fermata rappresenta la distanza totale percorsa fino all'arrivo alla stazione, non la distanza tra una fermata e l'altra. L'attributo Posizione invece rappresenta la posizione della stazione nell'ordine delle fermate di una Tratta.
I biglietti del tipo Business sono i più costosi, gli unici che offrono servizi e che quindi presenteranno il valore TRUE.
La tabella che segue riassume tutte le entità individuate dall'analisi dei requisiti e rappresentate nel diagramma E-R, con i relativi attributi:

Entità	**Descrizione**	**Attributi**	**Identificatore**
Stazione	Stazione ferroviaria	*Nome*, *Città*, *Regione*	*Nome*
Treno	Mezzo di trasporto	*Codice*, *Modello*, *PostiBusiness*, *PostiC1*, *PostiC2*	*Codice*
Regionale	Treno regionale		
Alta Velocità	Treno ad alta velocità		
Tratta	Inizio e fine percorso	*ID*, *Distanza*, *Durata*	*ID*
Utente	Utente del sistema	*CF*, *Nome*, *Cognome*, *Email*	*CF*
Controllore	Controllore treno		
Biglietto	Biglietto per tratta	*ID*, *Costo*	*ID*
Business	Biglietto business	*Servizi*	
Prima classe	Biglietto prima classe		
Seconda classe	Biglietto seconda classe		

(a) Entità

Ecco ora invece la tabella che riassume tutte le relazioni individuate:

Relazione	Descrizione	Componente	Attributi
Partenza	Stazione di partenza di una tratta	Stazione, Tratta	*Orario*, *Binario*
Arrivo	Stazione di arrivo di una tratta	Stazione, Tratta	*Orario*, *Binario*
Fermata	Stazione in cui una tratta ha una fermata	Stazione, Tratta	*Orario*, *Dist.Percorsa*, *Binario*, *Posizione*
Percorre	Un treno percorre una tratta	Treno, Tratta	*Data*, *Orario_Inizio*
Viaggio	Viene creato un biglietto relativo ad una tratta	Biglietto, Tratta	*Data_Partenza*
Acquista	Un utente acquista un biglietto	Utente, Biglietto	*Data*
Verifica	Un controllore verifica la validità del biglietto	Controllore, Biglietto	*Validità*, *Data*

(b) Relazioni

4 Progettazione Logica

In questa sezione viene illustrato il processo di "traduzione" dello schema concettuale in uno schema logico, con l'obiettivo di rappresentare i dati in modo preciso ed efficiente.
Si procede con l'eliminazione delle generalizzazioni di treno, utente e biglietto e infine viene presentato il diagramma ristrutturato, con una descrizione delle modifiche che sono state apportate al diagramma originale.

4.1 Eliminazioni delle generalizzazioni

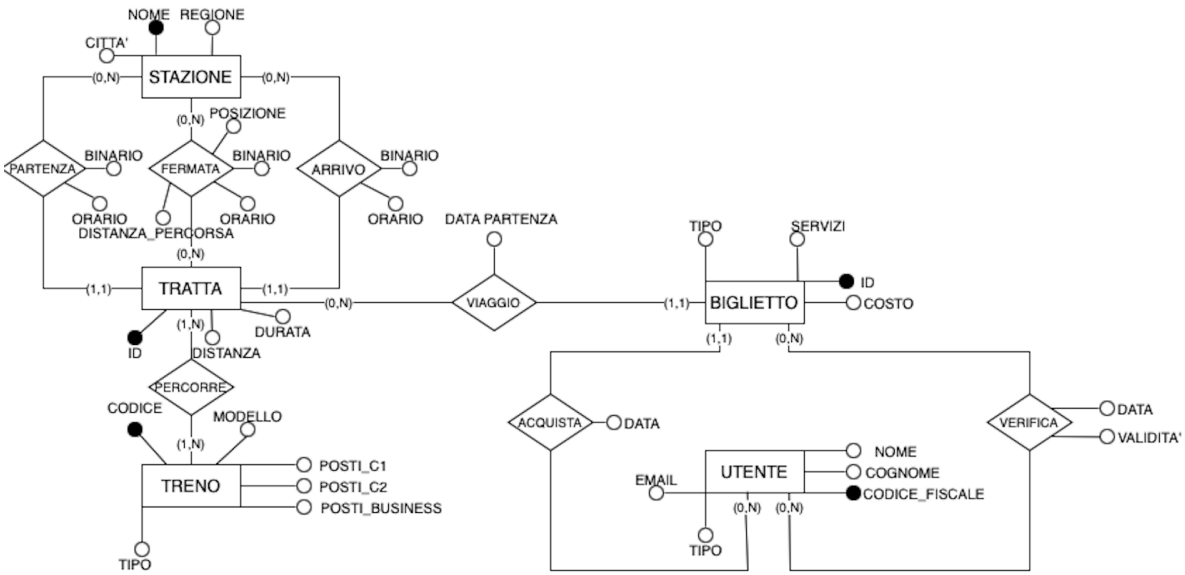
Le generalizzazioni descritte in sezione 3 vengono eliminate attraverso una ristrutturazione dello schema concettuale, con l'obiettivo di semplificare la successiva implementazione del modello relazionale e ridurre la presenza di valori nulli. Le modifiche vengono applicate come segue:

UTENTE la generalizzazione parziale ed esclusiva di UTENTE è costituita dalle superclassi CLIENTE, con relazione molti a uno per l'acquisto di un biglietto e CONTROLLORE, con relazione molti a molti per il controllo dei biglietti. Le 2 classi derivate non differiscono per attributi ma solo per le 2 relazioni appena descritte; inoltre, gli accessi sono contestuali, quindi si è deciso di accorpare le classi figlie alla classe padre inserendo un nuovo attributo "Tipo" che potrà assumere valori "Cliente" o "Controllore"

BIGLIETTO la generalizzazione totale ed esclusiva di BIGLIETTO costituita dalle superclassi PRIMACLASSE, SECONDACLASSE e BUSINESS, le quali sono costituite da un costo differente per ognuna e aggiunta di servizi per la classe BUSINESS. Si è quindi deciso di accorpare le classi figlie alla classe padre inserendo un nuovo attributo "Tipo"

che potrà assumere valori "prima","seconda","business" per l'identificazione della superclasse e l'aggiunta di un attributo "Servizi" che deve essere FALSE se l'attributo "Tipo" non è "Business".

TRENO la generalizzazione parziale ed esclusiva TRENO è costituita dalle superclassi REGIONALE ed ALTA VELOCITÀ, le quali non differiscono per attributi e non presentano relazioni; di conseguenza, la scelta migliore è quella di accorpare le classi figlie a quella padre, aggiungendo come visto precedentemente un attributo "Tipo" che potrà assumere valori "regionale" o "alta velocità".



4.2 Schema relazionale

Lo schema relazionale è rappresentato a seguire, dove l'asterisco dopo il nome degli attributi indica quelli che ammettono valori nulli.

- Stazione(<u>Nome</u>, Citta, Regione)
- Treno(<u>Codice</u>, Modello, Posti_Classe1, Posti_Classe2, Posti_Business, Tipo)
- Utente(<u>Codice_Fiscale</u>, Nome, Cognome, Email, Tipo)
- Tratta(<u>ID</u>, Distanza, Durata, Stazione_Partenza, Binario_Stazione_Partenza, Orario_Stazione_Partenza, Stazione_Arrivo, Binario_Stazione_Arrivo, Orario_Stazione_Arrivo)
  - Tratta.Stazione_Partenza → Stazione.Nome
  - Tratta.Stazione_Arrivo → Stazione.Nome
- Fermata(<u>Stazione_Nome, Tratta_ID</u>, Distanza_Percorsa, Posizione, Binario, Orario)
  - Fermata.Stazione_Nome → Stazione.Nome
  - Fermata.Tratta_ID → Tratta.ID

- Percorre(Tratta_ID, Codice_Treno, Data_Percorrenza, Orario_Inizio, Durata)
- Biglietto(ID, Servizi, Tipo, Data_Acquisto, Costo, Codice_Fiscale, Tratta_ID)
  - Biglietto.Codice_Fiscale → Utente.Codice_Fiscale
  - Biglietto.Tratta_ID → Tratta.ID
- Verifiche(CF_Controllore, ID_Biglietto, Data_Verifica, Valido)
  - Verifiche.CF_Controllore → Utente.Codice_Fiscale
  - Verifiche.ID_Biglietto → Biglietto.ID

5 Implementazione in PostgreSQL e Definizione delle query

Il file `ferrovia.sql` contiene il codice SQL necessario per la creazione e il popola- mento delle tabelle del database. Questo file include inoltre una serie di query per l'estrazione dei dati e un indice creato specificamente per migliorare le prestazioni di una di queste interrogazioni.

5.1 Definizione delle query

Di seguito vengono presentate e descritte le query con i relativi output generati e viene motivato l'utilizzo dell'indice proposto.

Query 1 Codice e numero di verifiche svolte da controllori a biglietti con costo maggiore di 50

```sql
SELECT V.cf_controllore, COUNT(*) as NumVerifiche
FROM verifiche V, biglietto B
WHERE V.id_biglietto = B.id
AND B.costo > 50 GROUP BY V.cf_controllore
```

Di seguito un estratto dell'output:

	cf_controllore character (16)	numverifiche bigint
1	000000000000CF99	2
2	000000000000CF98	5
3	000000000000CF93	6
4	000000000000CF94	6
5	000000000000CF91	9
6	000000000000CF92	10
7	00000000000CF100	5
8	000000000000CF96	6
9	000000000000CF97	6
10	000000000000CF95	5

Total rows: 10 Query complete 00:00:00.079

Query 2 Nome e cognome degli utenti che hanno acquistato almeno tre biglietti

```
SELECT U.nome, U.cognome , count(*) as NumBiglietti
FROM biglietto B, utente U
WHERE B.codice_fiscale = U.codice_fiscale
GROUP BY U.codice_fiscale
HAVING COUNT(*) > 2
```

Estratto dell'output:

5	Nome048	Cognome048	3
6	Nome021	Cognome021	3
7	Nome029	Cognome029	3
8	Nome078	Cognome078	3
9	Nome022	Cognome022	3
10	Nome067	Cognome067	3
Total rows: 90	Query complete 00:00:00.074		

Query 3 Codici dei treni che hanno percorso tratte che partono da Milano Centrale

```
SELECT P.codice_treno
FROM percorre P, tratta T
WHERE P.tratta_id = T.id
AND T.stazione_partenza = 'Milano Centrale'
```

Output:

	codice_treno character varying (8)
1	TR100

Query 4 Media del costo dei biglietti di tratte con distanze superiori ai 300km

```
SELECT avg(B.costo) AS MediaCosto
FROM biglietto B, tratta T
WHERE B.tratta_id = T.id AND T.distanza > 300
```

Estratto dell'output:

	mediacosto numeric
1	42.5000000000000000

Query 5 Numero di biglietti acquistati nelle varie giornate, con costo maggiore di 30 per tratte percorse da treni regionali

```sql
SELECT data_acquisto, COUNT(*) as NumBigliettiComprati
FROM biglietto B, tratta T, percorre P, treno TR
WHERE B.tratta_id = T.id AND T.id = P.tratta_id
AND TR.codice = P.codice_treno
AND TR.tipo = 'regionale' AND costo > 30
GROUP BY data_acquisto
```

Output:

	data_acquisto date	numbiglietticomprati bigint
1	2025-05-01	9
2	2025-05-02	18
3	2025-05-03	18
4	2025-05-04	18
5	2025-05-05	18

Query 6 Posti Businness, Classe1 e Classe2 disponibili in una tratta (parametro) di una determinata data (parametro)

```sql
WITH min_posti AS (
    SELECT
        p.tratta_id,
        MIN(t.posti_classe1) AS min_classe1,
        MIN(t.posti_classe2) AS min_classe2,
        MIN(t.posti_business) AS min_business
    FROM
        percorre p
    JOIN
        treno t ON p.codice_treno = t.codice
    WHERE
        p.tratta_id = 1
    GROUP BY
        p.tratta_id
),
biglietti_contati AS (
    SELECT
        b.tratta_id,
        COUNT(*) FILTER (WHERE tipo = 'prima classe' AND data_partenza = '2025-05-01') AS prenotati_classe1,
        COUNT(*) FILTER (WHERE tipo = 'seconda classe' AND data_partenza = '2025-05-01') AS prenotati_classe2,
        COUNT(*) FILTER (WHERE tipo = 'business' AND data_partenza = '2025-05-01') AS prenotati_business
    FROM
        biglietto b
    WHERE
        b.tratta_id = 1
    GROUP BY
        b.tratta_id
)
SELECT
    m.tratta_id,
    (m.min_classe1 - COALESCE(b.prenotati_classe1, 0)) AS disponibili_classe1,
    (m.min_classe2 - COALESCE(b.prenotati_classe2, 0)) AS disponibili_classe2,
    (m.min_business - COALESCE(b.prenotati_business, 0)) AS disponibili_business
FROM
    min_posti m
LEFT JOIN
    biglietti_contati b ON m.tratta_id = b.tratta_id;
```

Output:

	tratta_id integer	disponibili_classe1 bigint	disponibili_classe2 bigint	disponibili_business bigint
1	1	100	200	50

Query 7 Utenti ai quali è stato verificato un bigletto non valido su tratte composte da fermata(parametro)

```sql
SELECT
    u.nome,
    u.cognome,
    u.codice_fiscale
FROM
    utente u
JOIN
    biglietto b ON u.codice_fiscale = b.codice_fiscale
JOIN
    verifiche v ON v.id_biglietto = b.id
WHERE
    v.valido = FALSE
  AND
    b.tratta_id IN (
        SELECT
            f.tratta_id
        FROM
            fermata f
        WHERE
            f.stazione_nome = 'Bologna Centrale'
    );
```

Output:

	nome character varying (20)	cognome character varying (20)	codice_fiscale [PK] character (16)
1	Nome001	Cognome001	0000000000000CF1
2	Nome007	Cognome007	0000000000000CF7
3	Nome011	Cognome011	000000000000CF11
4	Nome017	Cognome017	000000000000CF17
5	Nome021	Cognome021	000000000000CF21
6	Nome027	Cognome027	000000000000CF27
7	Nome031	Cognome031	000000000000CF31
8	Nome037	Cognome037	000000000000CF37
9	Nome041	Cognome041	000000000000CF41
10	Nome047	Cognome047	000000000000CF47

5.2 Creazione degli indici

Si suppone di voler ottimizzare la query 2, per la quale occorre considerare:

1. "Condzione di Join": `B.codice_fiscale = U.codice_fiscale`;
2. Group by sulla colonna *U.codice_fiscale*;

Per il punto 1 e 2, è opportuno creare due indici di hash su entrambe le colonne:

```sql
CREATE INDEX idx_biglietto ON biglietto USING HASH (codice_fiscale);
CREATE INDEX idx_utente ON utente USING HASH (codice_fiscale);
```

Ciò permette di fare il join tra le due tabelle usando i codici hash, rendendo il tutto più efficiente.

6 Applicazione Software

Il file `codice.c` contiene il codice C necessario per connettersi al database e visualizzare a schermo i risultati delle query presentate nella sottosezione 5.1. Una volta compilato ed eseguito il programma, verrà mostrato all'utente un menù dove potrà selezionare in base al numero inserito la query da eseguire:

1. Codice e numero di verifiche svolte da controllori a biglietti con costo maggiore di 50.
2. Nome e cognome degli utenti che hanno acquistato almeno tre biglietti.
3. Codici dei treni che hanno percorso tratte che partono da (parametro).
4. Media del costo dei biglietti di tratte con distanze superiori ai 300km.
5. Numero di biglietti acquistati nelle varie giornate, con costo maggiore di (parametro) per tratte percorse da treni regionali.
6. Posti Businness, Classe1 e Classe2 disponibili in una tratta (parametro) di una determinata data (parametro).
7. Utenti ai quali è stato verificato un bigletto non valido su tratte composte da fermata (parametro).

Se viene inserito il numero 0 il programma invece termina.
La query 2, la query 5, la query 6 e la query 7 sono state rese parametriche: nella 2 è possibile selezionare la stazione di partenza della tratta d'interesse, nella 5 è possibile selezionare il costo dei biglietti, nella 6 è possibile selezionare la tratta (id) e la data del viaggio in un formato 'yyyy-mm-gg' e nella query 7 è possibile inserire una fermata che dovrà appartenere alla tratta sulla quale è stato fatto un controllo non valido del biglietto.